Operating Systems

ASSIGNMENT 1

CO1: *Apply the concepts of operating system types, structures and system calls to demonstrate their functions in various computing environments – history of operating systems, mainframe, desktop, multiprocessor, distributed, clustered, real-time and handheld systems, OS structure, services, system calls, system design, CPU states, memory hierarchy and I/O systems.*

Assignment Title: Evaluate and Justify the Operating-System Design of a Smart Hospital (Total Marks - 50)

Nature of Assignment: Individual
Duration: 1 Week
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Operating Systems for a Smart Hospital

A 500-bed hospital is replacing its computer systems. It needs: bedside patient monitors that must raise an alarm within 10 milliseconds of an abnormal reading; nurses' handheld tablets; doctors' desktop workstations; a central database server for patient records that must never stop; an imaging server that processes MRI scans using many processors; and a link to two partner hospitals that share records.

The IT team has proposed using one general-purpose operating system everywhere. The clinical staff are concerned that alarms may be delayed, the database may stop during updates, and the tablets' batteries will not last a shift.

The number of devices may grow:

1 ward (40 monitors, 10 tablets) → 1 hospital (500 monitors, 120 tablets, 80 workstations) → a group of 3 hospitals sharing records

You are required to evaluate the requirements of every part of the system, judge which type and structure of operating system suits each, examine how programs obtain OS services through system calls, and defend a complete operating-system plan for the hospital.

Design Challenge:

The hospital is adding remote patient monitoring at home and a cloud-based records service.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Evaluate the Requirements of Each Computing Environment 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate what each part of the hospital needs from its operating system.

Discuss:

- Response-time, reliability, throughput, battery and security requirements of each device
- Which requirements conflict (for example: fast response and many users on one machine)
- Which requirements are critical to patient safety
- How the lessons from the history of operating systems (batch, time-sharing, personal, mobile) apply to each environment

You are expected to **rank the requirements of each environment and justify the ranking**. **Alternative Concept and Complexity Analysis**

If the whole hospital used a single general-purpose operating system on every device, instead of a different type of OS for each environment, what would the solution be? Evaluate the worst-case alarm response time (unbounded scheduling delay versus a guaranteed deadline), the memory needed on each device, and the administration effort.

Evidence Expected

A ranked requirement table for every environment with a justification for each ranking

Method. Each environment was analysed for five requirements – response time, reliability, throughput, battery and security – and the requirements were ranked 1 (most important) to 5 (least important). The ranking is justified by what happens to a patient, a clinician or the hospital if that requirement fails.

1.1 Ranked requirement table

Environment	Ranking (1 = most important)	Justification
Bedside patient monitor (embedded, hard real-time)	1 Response time 2 Reliability 3 Security 4 Battery 5 Throughput	An alarm must sound within 10 ms; a late alarm is a patient-safety failure, so the deadline is rank 1. A monitor that hangs gives no alarm at all, so reliability is rank 2. Data rate is tiny (a few hundred samples per second), so throughput is last. Battery matters only for the backup battery during power cuts.
Nurse handheld tablet (mobile, soft real-time)	1 Battery 2 Security 3 Reliability 4 Response time 5 Throughput	The tablet must last a full 8–12 h shift without charging. It leaves the building and holds patient data, so loss or theft makes security rank 2. A UI delay of 100–200 ms is annoying but not dangerous (soft deadline).
Doctor desktop workstation (personal / time-sharing)	1 Security 2 Response time 3 Reliability 4 Throughput 5 Battery	Doctors open many records from many accounts, so authentication and access control matter most. Interactive use needs a response under about 100 ms. Mains powered, so battery is irrelevant.
Central database server (mainframe-class / clustered)	1 Reliability (availability) 2 Security 3 Throughput 4 Response time 5 Battery	The database "must never stop": downtime blocks every ward. Records are confidential, so security is rank 2. Hundreds of concurrent transactions make throughput important; a 10–50 ms query time is acceptable (soft).
Imaging server (multiprocessor, batch-like)	1 Throughput 2 Reliability (data integrity) 3 Security 4 Response time 5 Battery	MRI reconstruction is a long, CPU-heavy batch job: the number of scans per hour decides success. A corrupted scan is a clinical risk. Response time is minutes, not milliseconds.

Environment	Ranking (1 = most important)	Justification
Partner-hospital link (distributed)	1 Security 2 Reliability 3 Throughput 4 Response time 5 Battery	Records cross organisational borders over untrusted networks: authentication, encryption and audit are mandatory (privacy law). Messages must not be lost or duplicated. Delay of seconds is acceptable.

1.2 Conflicting requirements

- **Fast response vs many users on one machine:** a time-sharing scheduler (fair share, e.g. CFS) gives every user a slice, so one user's work can delay another by many milliseconds. A monitor needs a fixed-priority pre-emptive scheduler that always runs the alarm task first – these two goals cannot be met by one policy on one machine.
- **Throughput vs response time:** the imaging server wants long time slices and big batches (less context switching); interactive workstations want short slices.
- **Reliability vs frequent updates:** security patches require reboots, but the database must never stop – solved by a cluster with fail-over (patch one node at a time).
- **Battery vs performance:** the tablet saves energy by sleeping the CPU and radio, which adds wake-up latency.
- **Security vs response time:** encryption and access checks cost CPU cycles; on the monitor they must be cheap enough not to threaten the 10 ms deadline.
- **Small size vs rich services:** a general-purpose kernel with a file system, GUI and networking is too large and too unpredictable for a monitor.

1.3 Requirements critical to patient safety

- Monitor alarm deadline (≤ 10 ms) and monitor reliability – a missed alarm can cost a life.
- Database availability and integrity – wrong or unavailable allergy or medication data is dangerous.
- Imaging data integrity – a corrupted scan can lead to a wrong diagnosis.
- Security of records is a legal/ethical requirement, but it is not an immediate life-safety issue, so it ranks below response time and reliability for the monitor.

1.4 Lessons from the history of operating systems

Generation	Lesson	Applies to
Time-sharing (1960s–70s)	Many users share one machine through fast switching (time quantum), memory protection and file protection.	Database server and doctor workstations: many simultaneous users.

Generation	Lesson	Applies to
Personal computers (1980s–)	One user, good interactive response, simple protection, later networked.	Doctor workstations.
Real-time systems	Correctness depends on meeting a deadline; small, predictable kernels.	Bedside monitors.
Mobile / handheld (2000s–)	Power management, touch UI, app sandboxing, limited memory.	Nurse tablets.
Distributed / cloud	Resources spread over a network; need naming, security and consistency.	Partner link, later the cloud records service (Task 12).

Alternative Concept and Complexity Analysis – One general-purpose OS everywhere

Alternative: run one general-purpose OS (e.g. a standard Linux or Windows) on every device instead of a different OS type per environment.

Worst-case alarm response	RTOS: L = T_interrupt + T_context + T_ISR ≈ 20 µs + 10 µs + 50 µs < 1 ms. A guaranteed upper bound, below 10 ms.	Fair-share scheduler with long non-pre-emptible kernel sections, page faults, swap and disk I/O: delay can reach tens or hundreds of ms and has no guaranteed bound.	Bounded O(1) fixed-priority decision vs O(log n) CFS decision with unbounded wait.
Memory per monitor	FreeRTOS kernel ≈ 6–12 KB; complete firmware < 1 MB.	Minimum realistic Linux/Windows image: hundreds of MB of RAM and GBs of storage.	Space O(services) – 3–4 orders of magnitude difference.
Administration	4 OS images / patch streams, but each is small and independent.	1 image and 1 patch stream, but every patch affects the monitors and the database together and forces re-validation of safety functions.	Effort ≈ O(k) images vs O(1) images; risk grows with blast radius.

Conclusion: the single-OS plan fails the one requirement that matters most for safety – a guaranteed alarm deadline – so it is rejected for the monitors. It may be acceptable for workstations, where administration simplicity is more valuable.

Evidence provided: *ranked requirement table with justification for every environment (1.1), conflicts (1.2), safety-critical items (1.3), history (1.4).*

TASK 2 – Judge the Type of Operating System for Each Environment **3 Marks**

Bloom's Level: K5 – Evaluate

Compare the types of operating systems for the hospital.

Discuss:

- Mainframe, desktop, multiprocessor, distributed, clustered, real-time (hard and soft) and handheld systems

- Which type best fits the monitors, tablets, workstations, database server, imaging server and partner link
- Symmetric versus asymmetric multiprocessing for the imaging server
- A clustered system with fail-over for the database server

Alternative Concept and Complexity Analysis

If the database server used a single large multiprocessor machine instead of a cluster of smaller machines with fail-over, what would the solution be? Evaluate the availability (a single point of failure versus fail-over within seconds), the cost, and how throughput grows with the number of processors (time) and the memory available (space).

Evidence Expected

A justified choice of operating-system type for each environment, with the rejected alternatives and reasons.

2.1 Types of operating systems compared

Type	Key idea	Strength	Weakness for hospital
Mainframe	Batch + time-sharing, huge I/O capacity, very high reliability	Reliability, throughput, hot-swappable components	Cost; heavy for small devices
Desktop	One interactive user, GUI, many applications	Usability, application support	No timing guarantee
Multiprocessor (SMP / AMP)	Several CPUs share memory under one OS	Throughput for parallel work	Lock contention; shared memory is a single failure domain
Distributed	Independent machines cooperate over a network, appear as one system	Resource sharing, scalability, location transparency	Network failures, security, consistency
Clustered	Tightly coupled nodes with shared storage or replication and fail-over	High availability, load sharing	Cost of duplicated hardware, split-brain risk
Real-time (hard)	Missing a deadline is a failure; bounded latency	Predictability	Small feature set; lower throughput
Real-time (soft)	Deadlines are goals; occasional misses tolerated	Good multimedia and UI response	No hard guarantee
Handheld	Small screen, battery, limited memory, touch	Power management, app sandbox	Limited resources and storage

2.2 Chosen type for each environment, with rejected alternatives

Environment	Chosen OS type	Rejected alternatives and reasons
Bedside monitor	Hard real-time OS (e.g. QNX or FreeRTOS) with interrupt-driven I/O	Desktop/general-purpose: unbounded latency. Soft real-time: an alarm that is "usually" on time is not acceptable. Handheld OS: designed for apps and power saving, not deadlines.

Environment	Chosen OS type	Rejected alternatives and reasons
Nurse tablet	Handheld OS (Android / iOS style) with soft real-time UI	Desktop OS: too large, short battery life (Task 4). Hard RTOS: no rich app environment, no secure app ecosystem.
Doctor workstation	Desktop OS (Linux / Windows) joined to a central directory	Handheld: screen and input too limited. RTOS: no multi-application GUI.
Database server	Clustered system (2–3 nodes, shared or replicated storage, fail-over) on a server-grade Linux	Single machine: single point of failure. Mainframe: available but too costly for a 500-bed hospital. Desktop OS: not built for 24×7 server load.
Imaging server	Multiprocessor (SMP) server OS, many cores, large RAM, batch scheduling	AMP: a master CPU becomes a bottleneck and single point of failure. Cluster of small nodes would need to split every scan across the network.
Partner-hospital link	Distributed system over TLS/VPN with a standard distributed file/record protocol	Shared single machine: impossible across organisations. Unencrypted file sharing: violates privacy.

2.3 Symmetric vs asymmetric multiprocessing for the imaging server

Aspect	Symmetric (SMP)	Asymmetric (AMP)
Structure	Every CPU runs the OS and applications; one scheduler queue (or per-CPU queues)	One master CPU runs the OS; slave CPUs only run assigned work
Throughput	Scales with cores until locks and memory bandwidth limit it	Master saturates and caps the scale
Reliability	If a core fails the OS continues on the others	Master failure stops the system
Complexity	Needs locking of kernel data structures	Simple – only the master touches kernel data

Decision: SMP. MRI reconstruction splits into independent slices, so all cores can run the same code, and no master CPU becomes a bottleneck.

2.4 Clustered database server with fail-over

Two (or three) database nodes monitor each other with heartbeat messages. One node is active; the other is a hot standby with the same data (shared storage or synchronous replication). If the heartbeat stops for a few seconds, the standby takes over the service IP address and database role; clients reconnect. Patching is done one node at a time, so the database never stops.

Alternative Concept and Complexity Analysis – Single large multiprocessor instead of a cluster

Criterion	Cluster with fail-over	One large SMP machine
Availability	Both nodes must be down together: unavailability $\approx (1 - a)^2$ plus fail-over time. With a = 99.9% per node: $\approx 10^{-6}$ plus ≈ 30 s fail-over per failure → about 99.999% (a few minutes a year).	Single point of failure: availability = a = 99.9% → about 8.8 hours of downtime per year; every kernel update needs a reboot.
Cost	Two or three mid-range servers + shared storage / replication network.	One very large machine: price rises faster than linearly with CPUs and RAM; one spare would still be needed for availability.

Criterion	Cluster with fail-over	One large SMP machine
Throughput growth (time)	Add nodes (scale-out); reads scale well, writes need replication.	Amdahl's law: speed-up S(N) = 1 / ((1 − p) + p/N).
Memory (space)	RAM per node is limited; data is partitioned or duplicated (2× storage).	One large shared memory pool (up to several TB) with NUMA effects.

Amdahl's-law throughput growth (p = 0.95 parallel fraction):

Processors N	1	2	4	8	16	32	64	Limit
Speed-up	1.00	1.90	3.48	5.93	9.14	12.55	15.42	20.0

Doubling processors from 32 to 64 adds only 23%: time cost grows O(1/N) but flattens at the serial fraction. Space: a single SMP has O(M) uniform memory, the cluster has k × M_node but stores each datum r times. **Verdict:** the cluster is chosen because availability matters more than raw single-machine scaling.

Evidence provided: *justified OS type per environment with rejected alternatives (2.2), SMP/AMP comparison (2.3), cluster fail-over (2.4).*

TASK 3 – Evaluate Operating-System Structures **4 Marks**

Bloom's Level: K5 – Evaluate

Evaluate the possible structures of the operating system.

Discuss:

- Simple (monolithic), layered, microkernel, modular (loadable kernel modules) and hybrid structures
- The effect of each structure on reliability, performance and ease of updating
- Which structure suits the patient monitors and which suits the database server
- How Linux, Windows, Android and a real-time OS such as FreeRTOS or QNX are structured

Alternative Concept and Complexity Analysis

If the bedside monitors used a monolithic kernel instead of a microkernel, what would the solution be? Evaluate the time cost of a service request (one system call versus message passing between servers), the memory taken by the kernel, and the effect of a faulty driver on the whole system.

Evidence Expected

A comparison of structures against the hospital's requirements, with a defended choice for two environments.

3.1 Structures compared against hospital requirements

Structure	Description	Reliability	Performance	Ease of updating
Simple / monolithic	Whole OS in one kernel address space; all services call each other directly	Low – any bug in any driver can crash the kernel	Highest – a service is one function call	Poor – rebuild/reboot the whole kernel
Layered	OS built as layers 0…N; each uses only the layer below	Good – easy to verify layer by layer	Lower – every request crosses layers	Moderate
Microkernel	Only scheduling, IPC and memory management in kernel; drivers, file system, network run as user-space servers	Highest – a failed server is restarted without a crash	Lower – services need message passing (extra context switches)	Excellent – replace a server while running
Modular (loadable kernel modules)	Small monolithic core + modules loaded at run time	Medium – modules still run in kernel mode	High – no message passing	Good – load/unload a module without reboot
Hybrid	Monolithic-style kernel with microkernel ideas (some services in user space)	Medium–high	High	Good

3.2 How real systems are structured

System	Structure	Notes
Linux	Monolithic kernel with loadable kernel modules	Drivers, file systems and network stack run in kernel mode; modules can be inserted with insmod/modprobe.
Windows (NT)	Hybrid	Microkernel-style core plus an "executive" (I/O manager, memory manager, security reference monitor) and drivers in kernel mode; subsystems in user mode.
Android	Layered on top of a Linux kernel	Linux kernel → hardware abstraction layer → native libraries and Android Runtime → application framework → apps.
FreeRTOS	Minimal library kernel	Scheduler, queues, semaphores and timers linked with the application in one address space (no MMU needed); a few KB.
QNX Neutrino	True microkernel	Kernel does scheduling, IPC, interrupts and timers; drivers and file systems are user processes; used in medical and automotive devices.

3.3 Defended choice for two environments

Patient monitor → microkernel real-time OS (QNX-type; FreeRTOS on small microcontrollers). The kernel is tiny and predictable, so the worst-case latency can be calculated. A faulty driver (e.g. the display driver) runs outside the kernel and can be restarted in milliseconds while the alarm task keeps running. Extra service layers that would add delay

are simply not present.

Database server → Linux (monolithic kernel with loadable modules). Performance of file and network I/O is the priority, and the hardware is standard, with well-tested drivers. The risk of a driver crash is handled at cluster level: a failed node is replaced by the standby (Task 2). Modules allow adding a storage driver without rebooting.

Alternative Concept and Complexity Analysis – Monolithic kernel on the monitors instead of microkernel

Criterion	Microkernel (chosen)	Monolithic kernel	Complexity
Time cost of one service request	Client sends message to server and waits for reply: 2 mode switches + 2 context switches ≈ 1–5 µs; a request through k servers costs k messages.	One system call: one mode switch ≈ 0.1–0.3 µs (we measured ≈ 0.28 µs per read/write in Task 6).	Both O(1) per request but constant factor 3–10× higher for messages; O(k) for chained servers.
Kernel memory	Kernel ≈ tens to a few hundred KB (servers extra).	Typical Linux kernel image + modules: several MB to tens of MB.	Space O(core) vs O(core + all drivers).
Effect of a faulty driver	Driver is a user process: restarted; monitor keeps raising alarms.	Driver runs in kernel mode: kernel panic, whole monitor stops until reboot (tens of seconds).	Fault isolation: O(1) component vs whole system.

The 3–10× slower request is irrelevant because monitor workloads are tiny and the total path is still far below 10 ms; the fault-isolation benefit is decisive.

Evidence provided: *structure comparison against requirements (3.1), real systems (3.2), defended choices (3.3).*

TASK 4 – Evaluate the Services an Operating System Must Provide 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate the services each environment needs.

Discuss:

- User interface, program execution, I/O operations, file systems, communication, error detection, resource allocation, accounting, protection and security
- Which services can be removed from the bedside monitor to make it smaller and faster
- Which services are essential for the database server
- How the OS protects one patient's records from another user's programs

Alternative Concept and Complexity Analysis

If the tablets used a full desktop operating system with every service enabled instead of a reduced handheld OS, what would the solution be? Evaluate the memory and storage used by the

extra services (space), the start-up time and the battery life (time).

Evidence Expected

A service table for each environment with justified inclusions and exclusions.

4.1 Service table (Y = include, R = reduced, N = remove)

Service	Bedside monitor	Nurse tablet	Doctor workstation	Database server	Imaging server
User interface	N (fixed display only)	Y (touch GUI)	Y (GUI)	R (command line / remote)	R (command line / remote)
Program execution	R (fixed set of tasks)	Y	Y	Y	Y
I/O operations	Y (sensor, display, alarm)	Y	Y	Y	Y (DMA)
File system	R (flash log) or N	Y (encrypted)	Y	Y (journaled)	Y (large volumes)
Communication	R (secure link to nurse station)	Y (Wi-Fi)	Y	Y	Y
Error detection	Y (watchdog, sensor fault)	Y	Y	Y (ECC, checksums)	Y (ECC, checksums)
Resource allocation	R (fixed priorities)	Y (power-aware)	Y	Y	Y
Accounting	N	R	R	Y (audit)	Y (job accounting)
Protection & security	Y (minimal, authenticated updates)	Y	Y	Y	Y

4.2 Services removed from the bedside monitor

The monitor can drop the GUI/window system, multi-user login, general file system, printing, package manager and accounting. Removing them makes the kernel smaller (less memory and less code to certify), faster (fewer background tasks, shorter worst-case paths) and safer (smaller attack surface).

4.3 Services essential for the database server

- **File system with journaling and caching** – consistent records after a crash.
- **Process/thread management and multiprocessor scheduling** – many concurrent transactions.
- **Resource allocation** – memory, I/O bandwidth and locks shared fairly.
- **Protection and security** – authentication, access control, auditing.
- **Error detection and recovery** – ECC, checksums, RAID, fail-over.
- **Communication** – network service for clients and the cluster heartbeat.
- **Accounting/auditing** – who accessed which record (legal requirement).

4.4 Protecting one patient's records from another user's programs

- **Authentication** identifies the user (password, smart card, MFA).

- **File permissions / ACLs:** owner–group–other bits and access-control lists decide who can read or write a record; checked by the kernel on every open().
- **User/kernel mode and memory protection:** a program cannot read another program's memory because the hardware (base/limit or page tables) rejects the access; only the kernel can touch the disk.
- **Least privilege and mandatory access control** (e.g. SELinux) confine the database process.
- **Encryption** of the disk and network, plus **audit logs** of every access.

Alternative Concept and Complexity Analysis – Tablets with a full desktop OS

Typical order-of-magnitude values (to be checked on the actual devices):

Criterion	Reduced handheld OS (chosen)	Full desktop OS with all services	Complexity
Storage (space)	≈ 6–10 GB base system	≈ 15–30 GB base system	O(number of installed services)
Idle RAM (space)	≈ 1–1.5 GB, apps suspended by the OS	≈ 2–4 GB, many background daemons	O(running services)
Start-up time	≈ 15–25 s	≈ 40–90 s	O(services started at boot)
Battery life (time)	10–14 h (aggressive sleep/doze, one foreground app)	4–6 h (daemons keep CPU and Wi-Fi awake)	Power ≈ Σ power of active services

A full desktop OS would not last a shift and would add attack surface. The reduced handheld OS is justified.

Evidence provided: *service table (4.1) with justifications (4.2–4.4).*

TASK 5 – Examine System Calls 4 Marks

Bloom's Level: K5 – Evaluate

Examine how a program obtains OS services.

Discuss:

- The categories of system calls (process control, file management, device management, information maintenance, communication, protection)
- What happens step by step when a program calls read(): library call, trap, mode switch, kernel work, return
- How parameters are passed to the kernel (registers, a block in memory, the stack)
- How strace shows the system calls made by a program

Alternative Concept and Complexity Analysis

If programs called the C library functions fopen, fread and fwrite instead of the open, read and write system calls directly, what would the solution be? Evaluate the number of system calls for a file of n bytes (about n / 4096 with library buffering versus n / b for a buffer of b bytes) and the extra buffer space used by the library.

Evidence Expected

A step-by-step explanation of a system call with strace evidence from a real program. 5.1 Categories of system calls

Category	Purpose	Linux examples
Process control	Create, run, end and wait for processes; allocate memory	fork, execve, wait, exit, brk, mmap
File management	Create, open, read, write, close, delete files	open, read, write, close, unlink, stat
Device management	Request/release devices, read/write them	ioctl, read, write on /dev files
Information maintenance	Get/set time, system data, attributes	getpid, time, uname, sysinfo
Communication	Pipes, shared memory, messages, sockets	pipe, socket, send, recv, shmget
Protection	Control access to resources	chmod, chown, umask, setuid

5.2 What happens when a program calls read()

1. **Library call.** The program calls read(fd, buf, n). This is a C-library wrapper, an ordinary function call in user mode.
2. **Prepare parameters.** The wrapper puts the system-call number (0 for read on x86-64) in register RAX and the arguments fd, buf, n in RDI, RSI, RDX.

3. **Trap.** The wrapper executes the `syscall` instruction. The CPU saves the user program counter and flags, **switches the mode bit to kernel mode** and jumps to the kernel's system-call entry point.
4. **Kernel work.** The kernel saves registers, validates the file descriptor and buffer address, uses the system-call number to index the system-call table, and runs the read handler. The virtual file system finds the file; if the data is in the page cache it is copied to the user buffer, otherwise a disk request is queued and the process moves to the **waiting** state while another process runs; the disk interrupt later makes it **ready** again.
5. **Return.** The result (bytes read, or −errno) is placed in RAX; the CPU executes `sysret`, restoring user mode and the saved program counter.
6. **Back in the library.** The wrapper converts a negative result to errno = … and returns −1, otherwise returns the byte count to the program.

5.3 How parameters are passed to the kernel

Method	How it works	Used by
Registers	Number and arguments are placed in CPU registers (fast, limited to about 6 arguments)	Linux x86-64 (RAX, RDI, RSI, RDX, R10, R8, R9)
Block in memory	Parameters are stored in a memory block; its address is passed in a register	Used when there are more parameters than registers (e.g. some Linux calls with struct arguments, older Solaris)
Stack	Program pushes parameters on the stack; the kernel pops them	Older UNIX/BSD conventions, 32-bit x86 variants

5.4 Evidence: strace of a real program

Program readdemo.c opens /etc/hostname, reads it with read() and writes it to the terminal. It was compiled with gcc -std=c11 -Wall -Wextra and run under strace (Ubuntu 24.04, gcc 13.3).

Task05/readdemo.c

```
#define _POSIX_C_SOURCE 200809L
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
int main(void)
{
   char buf[64];
   int fd = open("/etc/hostname", O_RDONLY);   /* file management */
   ssize_t n = read(fd, buf, sizeof buf);      /* the read() system call */
   write(STDOUT_FILENO, buf, (size_t)n);       /* device management (terminal) */
   close(fd);
   return 0;
}
```

Output

```
$ strace ./readdemo 2>&1 | grep -E 'execve|hostname|read\(3|write\(1|close\(3|exit_group'
execve("./readdemo", ["./readdemo"], 0x7ffc807f3b50 /* 21 vars */) = 0
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0"..., 832) = 832
openat(AT_FDCWD, "/etc/hostname", O_RDONLY) = 3
read(3, "vm\n", 64)                     = 3
write(1, "vm\n", 3vm
close(3)                                = 0
exit_group(0)                           = ?

$ strace -c ./readdemo >/dev/null     (summary line)
100.00    0.000000           0        33         1 total
```

Reading the trace: the first read(3, "\177ELF…", 832) is the dynamic loader reading the C library header (before main). The program's own calls are openat → returns descriptor 3, read(3, "vm\n", 64) = 3 bytes, write(1, …) to standard output, close(3) and exit_group(0). Even this tiny program makes 33 system calls, mostly for loading libraries and setting up memory.

Alternative Concept and Complexity Analysis – fopen / fread / fwrite instead of open / read / write

Criterion	Direct system calls with buffer b	C library (stdio) with buffering
Number of system calls for n bytes	⌈n / b⌉ read calls (b chosen by the programmer; b = 1 gives n calls)	≈ ⌈n / 4096⌉ (library buffer = file-system block size), independent of how many bytes the program asks for per fread
Extra buffer space	O(b) – the programmer's own buffer	O(1) per open stream: about 4 KB buffer + FILE structure (a few hundred bytes)
Time	T = (n/b)·c_sys + n·c_copy	T = (n/4096)·c_sys + n·c_copy + small cost of the extra copy from library buffer to program

Example for n = 100 MB: with b = 4096 the call count is 25,600; with b = 65,536 it is 1,600 (measured in Task 6). stdio therefore protects a naive program that reads one byte or one record at a time: the user program may issue millions of fread/fgetc calls, but only about n/4096 system calls reach the kernel. The cost is one extra memory copy and a few KB of memory per stream.

Evidence provided: *step-by-step explanation of read() (5.2) and real strace output (5.4).*

TASK 6 – Evaluate the Cost of System Calls 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the number of system calls affects performance.

Discuss:

- Why each system call costs much more than an ordinary function call
- How the buffer size changes the number of read() and write() calls when the database copies a file.
- How library buffering (fread, fwrite) reduces the number of system calls
- The best buffer size for the hospital's backup program, using your measurements

Complete the Code

Complete TODO 1 below using the read() and write() system calls, and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L #include
<stdio.h>
#include <fcntl.h> #include
<unistd.h>

/* Copy src to dst with the read() and write() system calls, using a buffer of bufsize bytes. Returns the number
   of read() calls made, or -1 on error. */
long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    int in = open(src, O_RDONLY); if (in < 0)
    return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644); if (out < 0)
    { close(in); return -1; }
    long calls = 0; ssize_t
    n;
    // TODO 1: Loop: read up to bufsize bytes into buf (count every read call in calls);
    //       stop when read returns 0 (end of file) or -1; write exactly the n bytes that
    //       were read

    close(in); close(out);
    return calls;
}

int main(void)
{
    FILE *f = fopen("records.dat", "w");                    /* a 10,000-byte test file */
    for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f); fclose(f);

    size_t sizes[] = {1, 512, 4096}; for (int i = 0; i < 3;
    i++)
        printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i], copy_file("records.dat",
                "copy.dat", sizes[i]));
    unlink("records.dat");
    unlink("copy.dat"); return 0;


}
```

Expected output:

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

Run the program under strace -c and compare the number of read() calls reported with the program's output.

Alternative Concept and Complexity Analysis

If the backup program used memory-mapped files (mmap) instead of read() and write() with a buffer, what would the solution be? Evaluate the number of system calls (a few versus about n / b), the copying between kernel and user space, and the memory used for very large files.

Evidence Expected

Measured system-call counts and times for different buffer sizes,with a justified recommendation.

6.1 Why a system call costs more than a function call

- A function call is a jump within user mode: about 1–2 ns (push return address, jump, return).
- A system call needs a mode switch (trap), saving and restoring registers, switching to the kernel stack, argument validation, a table dispatch and a return – hundreds of cycles. Security mitigations (page-table isolation, speculation barriers) add more.
- It disturbs the CPU caches, TLB and branch predictor, so the user program runs slower for a while after the call.
- **Measured:** copying an 8 MB sample with a 1-byte buffer took 4.698 s for 8,388,608 read + 8,388,608 write calls → **≈ 0.28 µs per system call**, roughly 200–300 times a function call.

6.2 Completed program (TODO 1 filled in)

Task06/copy_syscalls.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>


/* Copy src to dst with the read() and write() system calls, using a buffer of
   bufsize bytes. Returns the number of read() calls made, or -1 on error. */
long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    int in = open(src, O_RDONLY);
    if (in < 0) return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }
    long calls = 0;
    ssize_t n;
    /* TODO 1 - completed */
    while (1) {
        n = read(in, buf, bufsize);
        calls++;                    /* count every read() call */
        if (n <= 0) break;          /* 0 = end of file, -1 = error */
        if (write(out, buf, (size_t)n) != n) { calls = -1; break; }
    }
    close(in);
    close(out);
    return calls;
}
```

```
int main(void)
{
   FILE *f = fopen("records.dat", "w"); /* a 10,000-byte test file */
   for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);
   fclose(f);
   size_t sizes[] = {1, 512, 4096};
   for (int i = 0; i < 3; i++)
      printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i],
            copy_file("records.dat", "copy.dat", sizes[i]));
   unlink("records.dat");
   unlink("copy.dat");
   return 0;
}
```

Output

```
$ gcc -std=c11 -Wall -Wextra -o copy_syscalls copy_syscalls.c && ./copy_syscalls
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

The output is identical to the expected output. The count is ⌈10000 / b⌉ + 1: the last read() returns 0 (end of file) and is counted too (10000 + 1, 20 + 1, 3 + 1).

Comparison with strace -c

Output

```
$ strace -c -e trace=read,write,openat ./copy_syscalls
% time     seconds  usecs/call     calls    errors syscall
 52.72    0.010147           1     10027           write
 46.97    0.009039           0     10027           read
  0.31    0.000060           6         9         openat
100.00    0.019246           0     20063           total
```

The program printed 10001 + 21 + 4 = **10026** read() calls. strace reports **10027**: the one extra read() is made by the dynamic loader when it loads the C library (the 832-byte ELF header read seen in Task 5). The write count 10027 = 10000 + 20 + 3 data writes + 3 writes by fputc buffering when creating records.dat + 1 write of the printf output. The program and strace therefore agree.

Experiment: buffer size and the number of read() calls (the database copying a file)

Rule: reads = ⌈n / b⌉ + 1. For the 10,000-byte file: b = 1 → 10,001; b = 512 → 21; b = 4096 → 4. Doubling the buffer halves the number of calls – O(n / b).

6.3 Measurements: a 100 MB file

bench_copy.c copies a 100 MB file with each method and measures wall-clock time (Ubuntu 24.04 VM, 1 vCPU, gcc 13.3). The 1-byte case was measured on an 8 MB sample and scaled

×12.5 because 100 million single-byte calls take about a minute.

Method (buffer)	read() calls for 100 MB	Time (s)
read/write, 1 B	104,857,600 (8,388,608 for the 8 MB sample)	≈ 58.7 (estimated from 4.698 s for 8 MB)
read/write, 64 B	1,638,400	0.941
read/write, 512 B	204,800	0.248
read/write, 4096 B	25,600	0.177
read/write, 65,536 B	1,600	0.077
read/write, 1,048,576 B	100	0.086
fread/fwrite, 4096 B	25,600 library calls	0.117
mmap + memcpy	about 5–6 in total	0.131

Interpretation. Time falls steeply until about 64 KB because the per-call cost c_sys is paid n/b times: T(b) ≈ (n/b)·c_sys + n·c_copy. After about 64 KB the first term is negligible and the copy term n·c_copy dominates, so a larger buffer no longer helps; at 1 MB the buffer no longer fits comfortably in the CPU cache and time rises slightly (0.086 s). In this run fread/fwrite (0.117 s) was somewhat faster than raw 4 KB read/write (0.177 s); the difference is within the run-to-run variation seen on this VM, so re-measure it on your machine before drawing a conclusion.

Recommendation for the hospital backup program: use a buffer of **64 KB** (a multiple of the 4 KB page size; 128 KB–256 KB would perform similarly). It gives about 2.3× the speed of a 4 KB buffer, 3.2× that of 512 B, 12× that of 64 B and about 760× that of 1 byte, using only 64 KB of memory per copy stream, so many backups can run in parallel.

Task06/bench_copy.c (benchmark source)

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <time.h>
#include <sys/mman.h>
#include <sys/stat.h>

static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec+t.tv_nsec/1e9; }

long copy_rw(const char *s, const char *d, size_t bs)
{
```

```
    char *buf = malloc(bs);
    int in = open(s, O_RDONLY), out = open(d, O_WRONLY|O_CREAT|O_TRUNC, 0644);
    long calls = 0; ssize_t n;
    while ((n = read(in, buf, bs)) > 0) { calls++; if (write(out, buf, n) != n) break; }
    close(in); close(out); free(buf); return calls;
}
long copy_stdio(const char *s, const char *d, size_t bs)
{
    char *buf = malloc(bs);
    FILE *in = fopen(s,"rb"), *out = fopen(d,"wb"); size_t n; long calls=0;
    while ((n = fread(buf,1,bs,in)) > 0) { calls++; fwrite(buf,1,n,out); }
    fclose(in); fclose(out); free(buf); return calls;
}
void copy_mmap(const char *s, const char *d)
{
    int in = open(s, O_RDONLY), out = open(d, O_RDWR|O_CREAT|O_TRUNC, 0644);
    struct stat st; fstat(in,&st);
    if (ftruncate(out, st.st_size) != 0) return;
    char *a = mmap(NULL, st.st_size, PROT_READ, MAP_PRIVATE, in, 0);
    char *b = mmap(NULL, st.st_size, PROT_WRITE, MAP_SHARED, out, 0);
    for (off_t i = 0; i < st.st_size; i += 4096) { size_t c = st.st_size - i < 4096 ? st.st_size - i : 4096; __builtin_memcpy(b+i, a+i, c); }
    munmap(a, st.st_size); munmap(b, st.st_size); close(in); close(out);
}
int main(void)
{
    const char *src = "big.dat";
    size_t total = 100u<<20;
    FILE *f = fopen(src,"wb"); char blk[65536]; for (int i=0;i<65536;i++) blk[i]=i%251;
    for (size_t i=0;i<total;i+=sizeof blk)
        fwrite(blk,1,sizeof blk,f);
    fclose(f);
    printf("File size: %zu MB\n%-22s %12s %10s\n", total>>20, "method (buffer)", "read calls", "time (s)");
    size_t sizes[] = {1, 64, 512, 4096, 65536, 1048576};
    for (int i = 0; i < 6; i++) {
        if (sizes[i] == 1) { /* 100M single-byte syscalls is too slow: use 8 MB sample */
            FILE *g=fopen("small.dat","wb"); for(int k=0;k<128;k++) fwrite(blk,1,65536,g); fclose(g);
            double t=now(); long c=copy_rw("small.dat","o.dat",1); t=now()-t;
            printf("%-22s %12ld %10.3f   (8 MB sample, x12.5 = %.1f s est.)\n","read/write 1 B",c,t,t*12.5);
            unlink("small.dat"); continue; }
        double t=now(); long c=copy_rw(src,"o.dat",sizes[i]); t=now()-t;
        char nm[40]; snprintf(nm,sizeof nm,"read/write %zu B",sizes[i]);
        printf("%-22s %12ld %10.3f\n",nm,c,t);
    }
    double t=now(); long c=copy_stdio(src,"o.dat",4096); t=now()-t;
    printf("%-22s %12ld %10.3f  (fread calls; stdio uses 4096 B internal buffer)\n","fread/fwrite 4096 B",c,t);
    t=now(); copy_mmap(src,"o.dat"); t=now()-t;
    printf("%-22s %12s %10.3f\n","mmap + memcpy","~5 syscalls",t);
    unlink(src); unlink("o.dat"); return 0;
}
```

Output

```
$ ./bench_copy
File size: 100 MB
method (buffer)        read calls   time (s)
read/write 1 B          8388608     4.698   (8 MB sample, x12.5 = 58.7 s est.)
read/write 64 B         1638400     0.941
read/write 512 B         204800     0.248
read/write 4096 B         25600     0.177
read/write 65536 B         1600     0.077
read/write 1048576 B        100     0.086
fread/fwrite 4096 B       25600     0.117  (fread calls; stdio uses 4096 B internal buffer)
mmap + memcpy          ~5 syscalls     0.131
```

Alternative Concept and Complexity Analysis – Memory-mapped files (mmap) instead of read() / write()

```
/* pseudocode */
in  <- open(src);  out <- open(dst);  size <- fstat(in).size
ftruncate(out, size)
a <- mmap(NULL, size, PROT_READ,  MAP_PRIVATE, in,  0)
b <- mmap(NULL, size, PROT_WRITE, MAP_SHARED,  out, 0)
memcpy(b, a, size)          /* page faults load pages on demand */
munmap(a); munmap(b); close(in); close(out)
```

Criterion	read()/write() with buffer b	mmap
System calls	2 × ⌈n / b⌉ = O(n / b)	about 6–10 in total = O(1)
Copying between kernel and user space	Every byte copied kernel → user buffer and user buffer → kernel (2 copies)	File pages in the page cache are mapped directly; no kernel↔user copy for the input (here one user-level memcpy remains)
Page faults	None for the buffer	About n / 4096 minor page faults, each ≈ 0.5–1 µs
Memory for very large files	O(b) – constant	Virtual address space O(n), but physical memory is limited to what the page cache keeps; works for files larger than RAM through demand paging
Measured (100 MB)	0.077 s (64 KB), 0.177 s (4 KB)	0.131 s

mmap removes most system calls and one copy, but replaces them with page faults. In this run it was faster than a 4 KB buffer (0.131 s vs 0.177 s) but slower than a well-sized 64 KB buffer (0.077 s). For a sequential backup the tuned read/write buffer is therefore the best choice; mmap is attractive for random access to large files.

Evidence provided: *measured system-call counts and times for different buffer sizes with a justified recommendation(64KB).*

TASK 7 – Evaluate Process Control System Calls 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how a command shell on the database server starts and manages programs.

Discuss:

- What fork(), exec(), wait() and exit() each do
- How a shell runs a command, and how it obtains the command's exit status
- What a zombie and an orphan process are, and how the OS deals with them
- Why fork() followed by exec() is used instead of a single "create process" call

Complete the Code

Complete TODO 1 below using fork(), execlp() and waitpid(), and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L #include
<stdio.h>
#include <stdlib.h> #include
<unistd.h> #include <sys/wait.h>

int main(void)
{
        printf("shell: running job 1\n"); fflush(stdout);
                                                        /* empty the buffer before fork() */
        // TODO 1: Create a child with fork(). In the child, replace the program with
        //       execlp("expr", "expr", "6", "*", "7", (char *)NULL) and exit(127) if exec
        //       fails. In the parent, wait for the child with waitpid() and print "job 1 exit
        //       status N" using WEXITSTATUS


        printf("shell: running job 2\n"); fflush(stdout);
        pid = fork();
        if (pid == 0) _exit(3); waitpid(pid, &status, 0);
                                                        /* a child that fails with code 3 */

        printf("job 2 exit status %d\n", WEXITSTATUS(status)); return 0;

}
```

Expected output:

```
shell: running job 1 42
job 1 exit status 0 shell:
running job 2 job 2 exit
status 3
```

Explain why stdout is flushed before each fork(). Then remove the second fflush(stdout), change _exit(3) to exit(3), redirect the output to a file and explain why “shell: running job 2” now appears twice.

Alternative Concept and Complexity Analysis

If commands were run with posix_spawn() or with threads instead of fork() followed by exec(), what would the solution be? Evaluate the time to start 1,000 commands, the memory copied or shared (copy-on-write), and the isolation between commands.

Evidence Expected

A working program that runs commands like a shell, with process diagrams and a reasoned evaluation.

7.1 What fork(), exec(), wait() and exit() do

Call	Action
fork()	Creates a new process that is a copy of the caller (same code, data, open files). Returns the child's PID to the parent and 0 to the child. Memory is shared copy-on-write.
exec() family (execlp, execve …)	Replaces the calling process's program, stack and data with a new program; the PID and open files stay. Returns only on failure.
wait() / waitpid()	Parent sleeps until a child ends and collects its exit status; this removes the child's process-table entry.
exit() / _exit()	Ends the process, closes its files and passes an exit status to the parent. exit() first runs atexit handlers and flushes stdio buffers; _exit() does not.

7.2 How a shell runs a command and obtains its exit status

Process diagram

```
shell (parent)                    child
--------------                    -----
read command "expr 6 * 7"
pid = fork() ----------------------> (copy of shell, pid = 0)
waitpid(pid) [sleeps, state=waiting]   execlp("expr",...)  -> memory replaced
                                  expr prints 42, calls exit(0)
<------------ exit status 0 --------- child terminated
WEXITSTATUS(status) = 0 -> "job 1 exit status 0"
prompt for the next command
```

The shell forks, the child execs the program, and the shell waits. The status integer filled in by waitpid() encodes how the child ended; WEXITSTATUS(status) extracts the exit code (0 = success, non-zero = failure; 127 means "command not found/exec failed"). A shell stores it in $?.

7.3 Zombie and orphan processes

- **Zombie:** a child that has exited but whose parent has not yet called wait(). Its process-table entry (PID and exit status) remains so the parent can read the status. Many

zombies exhaust the process table. Fix: parent calls wait()/waitpid() or handles SIGCHLD; if the parent dies, init (PID 1) adopts and reaps them.

- **Orphan:** a child whose parent has exited while the child is still running. The OS re-parents it to init/systemd (PID 1), which later calls wait() when it ends, so orphans do not become permanent zombies.

7.4 Why fork() followed by exec() instead of one "create process" call

- Between fork() and exec() the child can change its own environment – redirect stdin/stdout (dup2), set up pipes, change directory, drop privileges, set limits – using ordinary system calls, so the process-creation call stays simple and every combination of setup is possible (this is how the shell implements > and |).
- fork() is cheap thanks to copy-on-write: pages are copied only if either process writes.
- Cost: fork() of a large process must copy its page tables, O(P) in the parent's mapped pages (measured below).

7.5 Completed program (TODO 1 filled in)

Task07/shell.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>


int main(void)
{
   int status;
   pid_t pid;
   printf("shell: running job 1\n");
   fflush(stdout); /* empty the buffer before fork() */


   /* TODO 1 - completed */
   pid = fork();
   if (pid < 0) { perror("fork"); return 1; }
   if (pid == 0) {
      execlp("expr", "expr", "6", "*", "7", (char *)NULL);
      _exit(127);              /* reached only if exec failed */
   }
   waitpid(pid, &status, 0);
   printf("job 1 exit status %d\n", WEXITSTATUS(status));


   printf("shell: running job 2\n");
   fflush(stdout);
   pid = fork();
   if (pid == 0) _exit(3); /* a child that fails with code 3 */
   waitpid(pid, &status, 0);
```

```
    printf("job 2 exit status %d\n", WEXITSTATUS(status));
    return 0;
}
```

Output

```
$ gcc -std=c11 -Wall -Wextra -o shell shell.c && ./shell
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

The output is identical to the expected output. expr 6 * 7 prints 42 and exits with status 0; job 2's child calls _exit(3) and the parent reports 3.

7.6 Why stdout is flushed before fork() – and the experiment

When stdout is a terminal it is line-buffered; when it is a file or pipe it is **fully buffered** – text stays in a buffer in the process's user-space memory until it fills, is flushed or the process exits normally. fork() copies that memory, so the child gets its own copy of any unflushed text. If the buffer is not emptied first, **both** processes eventually flush it and the text is printed twice (and possibly out of order). fflush(stdout) before fork() guarantees the buffer is empty at the moment of the copy.

Experiment: the second fflush(stdout) was removed, _exit(3) was changed to exit(3) and the output redirected to a file (shell_buggy.c).

Output

```
$ gcc -std=c11 -Wall -Wextra -o shell_buggy shell_buggy.c && ./shell_buggy > out.txt; cat out.txt
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Explanation: in the redirected run the line "job 1 exit status 0" and "shell: running job 2" are still sitting in the buffer when the second fork() is executed. The child inherits a copy of this buffer. Because it now ends with exit(3), the C library flushes the stdio buffers before exiting, so the child prints "job 1 exit status 0 / shell: running job 2" once, and then the parent prints the same buffered text again when it exits. With _exit(3) the child skips the flush, which is why the original program is correct. (Note: "shell: running job 2" appears twice, and so does the job 1 status line, because both were in the buffer.) On an interactive terminal stdout is

line-buffered, so the buffer is empty at fork() and the duplication does not happen – re-run on your own terminal to confirm; the test was run here with output captured through a pipe.

7.7 Evidence from a second experiment: scale of fork() cost

Task07/spawn_vs_fork.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <spawn.h>
#include <time.h>
#include <unistd.h>
#include <sys/wait.h>
extern char **environ;
static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec+t.tv_nsec/1e9; }
static double run_fork(int n)
{
   double t = now();
   for (int i = 0; i < n; i++) {
      pid_t p = fork();
      if (p == 0) { execl("/bin/true", "true", (char *)NULL); _exit(127); }
      waitpid(p, NULL, 0);
   }
   return now() - t;
}
static double run_spawn(int n)
{
   double t = now();
   for (int i = 0; i < n; i++) {
      pid_t p; char *argv[] = {"true", NULL};
      posix_spawn(&p, "/bin/true", NULL, NULL, argv, environ);
      waitpid(p, NULL, 0);
   }
   return now() - t;
}
int main(void)
{
   const int N = 1000;
   size_t sizes[] = {0, 256u << 20};          /* extra parent memory touched */
   for (int k = 0; k < 2; k++) {
      char *big = NULL;
      if (sizes[k]) { big = malloc(sizes[k]); memset(big, 1, sizes[k]); }
      double tf = run_fork(N), ts = run_spawn(N);
      printf("parent +%3zu MB: fork+exec %.0f ms (%.0f us each) | posix_spawn %.0f ms (%.0f us each)\n",
            sizes[k] >> 20, tf*1e3, tf*1e6/N, ts*1e3, ts*1e6/N);
      free(big);
   }
   return 0;
}
```

Output

```
$ ./spawn_vs_fork       (run 1)
parent +  0 MB: fork+exec 1076 ms (1076 us each) | posix_spawn 1094 ms (1094 us each)
parent +256 MB: fork+exec 5222 ms (5222 us each) | posix_spawn 1055 ms (1055 us each)
$ ./spawn_vs_fork       (run 2)
parent +  0 MB: fork+exec 1007 ms (1007 us each) | posix_spawn 1100 ms (1100 us each)
parent +256 MB: fork+exec 4132 ms (4132 us each) | posix_spawn 1179 ms (1179 us each)
```

Alternative Concept and Complexity Analysis – posix_spawn() or threads instead of fork() + exec()

Criterion	fork() + exec()	posix_spawn()	Threads (pthread_create)
Time to start 1,000 commands (measured)	≈ 1.0–1.1 s with a small parent; 4.1–5.2 s when the parent has touched 256 MB	≈ 1.1 s whatever the parent size	Threads run functions in the same program, they do not start other programs; creation ≈ 30 µs each (Task 10)
Memory copied or shared	Page tables copied (O(P) in the parent's pages); data pages shared copy-on-write until written	Child runs with a minimal new address space (vfork/CLONE_VM style) – O(1) regardless of parent size	Everything shared: one address space, heap, globals
Isolation between commands	Complete – separate address space, own fault domain	Complete – separate process	None – a bug or crash in one thread kills all
Flexibility	Any set-up code between fork and exec	Limited to the supported "file actions" and attributes	Not applicable to running arbitrary programs

Complexity: fork cost ≈ O(P) with the number of mapped pages P of the parent, so a large server shell is penalised; posix_spawn is O(1) in the parent's size. For 1,000 commands total time is 1000 × (c_fixed + c_P·P) vs 1000 × c_fixed. Threads are the cheapest but give no isolation, so they are unsuitable for running independent administrator commands on the database server.

Evidence provided: *working program, process diagram (7.2), measured evaluation (7.7).*

TASK 8 – Evaluate CPU States and Protection 3 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the hardware protects the hospital system.

Discuss:

- User mode and kernel mode, privileged instructions and the mode bit
- Interrupts, traps and the timer interrupt
- What would happen to the monitors if any program could disable interrupts

- How the hardware stops one program from reading another program's memory

Alternative Concept and Complexity Analysis

If the monitors used polling (checking the sensor in a loop) instead of interrupts, what would the solution be? Evaluate the worst-case delay before an abnormal reading is noticed (the polling interval versus the interrupt latency) and the CPU time and power wasted by polling.

Evidence Expected

A reasoned explanation of CPU modes, interrupts and protection, applied to the monitors.

8.1 User mode, kernel mode, privileged instructions and the mode bit

The CPU has a **mode bit** (on x86 the current privilege level in the code-segment register: ring 0 = kernel, ring 3 = user). In user mode a program may execute only ordinary instructions on its own memory. **Privileged instructions** – such as enabling/disabling interrupts (cli/sti), loading page-table registers, halting the CPU, and I/O port instructions – cause a trap if executed in user mode. The only way to enter kernel mode is through a controlled gate: a system call (trap), an interrupt or an exception, which also forces the jump to a fixed kernel address.

8.2 Interrupts, traps and the timer interrupt

Event	Source	Purpose / example
Interrupt	Hardware, asynchronous	Sensor data ready, disk finished, network packet arrived; CPU saves state and runs the interrupt service routine (ISR)
Trap / system call	Software, deliberate	Program asks the kernel for a service (read, write)
Exception	Software, error	Divide by zero, invalid memory access (page fault, segmentation fault)
Timer interrupt	Hardware timer	Fires every few ms; the kernel regains control, runs the scheduler and prevents a program from keeping the CPU forever

8.3 What if any program could disable interrupts?

On the monitors the sensor interrupt is what triggers the alarm path. A faulty or malicious program that executes cli and then loops would **stop the timer and the sensor interrupts**: the scheduler would never run, no other task would execute, and no alarm would sound – the 10 ms deadline would be missed indefinitely. It could also corrupt devices or other programs. This is why disabling interrupts is a privileged instruction; on the monitors even kernel code must keep interrupt-off sections to a few microseconds, because they directly add to the worst-case alarm latency.

8.4 How the hardware stops one program from reading another program's memory

- **Base and limit registers (simple scheme):** every address generated in user mode is checked: base ≤ address < base + limit; otherwise a trap goes to the OS.
- **Paging with a memory-management unit (MMU):** each process has its own page table that maps its virtual pages only to frames it owns. There is no entry for another process's frames, so the access is impossible; entries also carry read/write/execute and user/supervisor bits. A violation raises a page fault; the OS kills the process (segmentation fault).
- Changing the page-table base register is privileged, so a program cannot give itself access to another address space.
- On the cheapest monitor microcontrollers without an MMU, a memory-protection unit (MPU) provides a few protected regions for the same purpose.

Alternative Concept and Complexity Analysis – Polling instead of interrupts on the monitors

```
/* polling version (pseudocode) */
loop forever:
   v <- read_sensor()
   if v is abnormal: raise_alarm()
   sleep(T)              /* polling interval */


/* interrupt version */
on sensor_interrupt:  (ISR)  if value abnormal: raise_alarm()
main: idle (CPU halts with WFI until the next interrupt)
```

Criterion	Polling	Interrupts
Worst-case delay before an abnormal reading is noticed	Up to the polling interval T (an abnormality that occurs just after a poll waits a full T). To meet 10 ms with 2 ms processing, T ≤ 8 ms.	Interrupt latency only: typically 5–50 µs plus ISR time – a bound independent of how often readings change.
CPU time used	Each poll costs about c ≈ 2 µs. At T = 1 ms this is 0.2% of the CPU; at 100 kHz polling for 10 µs response it would be 20%; the CPU can never sleep.	CPU is used only when an event occurs; otherwise it sleeps.
Power	High and constant – CPU cannot enter low-power state.	Low – wait-for-interrupt sleep.
Complexity	Detection time O(T); work per second O(1/T)	Detection time O(1); work proportional to the number of events

There is a direct trade-off for polling: shortening the delay requires more polls and more CPU time and power. Interrupts give the lowest, bounded delay at the lowest cost, so they are

chosen for the monitors.

Evidence provided: *reasoned explanation of CPU modes, interrupts and protection applied to monitors.*

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems **4 Marks**

Bloom's Level: K5 – Evaluate

Evaluate how data moves between storage levels and devices.

Discuss:

- The levels of the memory hierarchy (registers, cache, main memory, SSD, hard disk, network storage) with typical access times and sizes
- Where MRI images, patient records and monitor readings should be kept
- Programmed I/O, interrupt-driven I/O and DMA, and which suits the MRI scanner
- The effect of caching on the speed of record look-up

Alternative Concept and Complexity Analysis

If the MRI scanner transferred images with interrupt-driven I/O (one interrupt per word) instead of DMA, what would the solution be? Evaluate the number of interrupts and the CPU time for a 500 MB image, and the memory buffers needed in each method.

Evidence Expected

A justified data-placement plan and I/O method for each device, using typical access times.

9.1 Levels of the memory hierarchy (typical values)

Level	Typical size	Typical access time	Persistence
Registers	< 1 KB	≈ 0.3 ns (1 cycle)	Volatile
L1 / L2 / L3 cache	32 KB – 64 MB	1 ns / 4 ns / 15 ns	Volatile
Main memory (DRAM)	8 GB – TBs	≈ 80–100 ns	Volatile
SSD (NVMe/SATA)	256 GB – 8 TB	≈ 20–100 µs	Non-volatile
Hard disk	1 – 20 TB	≈ 5–10 ms (seek + rotation)	Non-volatile
Network storage / cloud	PBs	≈ 0.5–100 ms (LAN to WAN)	Non-volatile

9.2 Data-placement plan

Data	Placement	Reason
Monitor readings (live)	Registers/SRAM and a ring buffer in main memory of the monitor; periodic flush to the database	Must be processed within 10 ms; RAM access ≈ 100 ns, a disk access (ms) would break the deadline.

Data	Placement	Reason
Active patient records	Database server: hot data in DRAM cache and on SSD; full database on mirrored SSD/RAID; replicated to the standby node	Frequent random access; SSD ≈ 100× faster than HDD for random reads.
MRI images (hundreds of MB each)	Imaging server NVMe SSD during processing; then hard-disk RAID / network storage for the archive; old studies on cheaper cloud storage	Large, sequential, rarely re-read after a few weeks; capacity and cost matter more than latency.
Backups and archives	Hard disks / network storage in another building	Capacity and durability.

9.3 Programmed I/O, interrupt-driven I/O and DMA

Method	How it works	CPU involvement	Suitable for
Programmed I/O (polling)	CPU repeatedly tests a status register, then moves each word	Busy for the whole transfer	Very simple, fast devices; rarely used
Interrupt-driven I/O	Device interrupts the CPU for each word/block; ISR moves it	Interrupted per word	Slow, sporadic devices (keyboard, monitor sensor)
DMA	CPU programs the DMA controller with source, destination and length; the controller moves the data and interrupts once at the end	Almost none during the transfer	Bulk data: disk, network, MRI scanner

Choice: MRI scanner → DMA (megabytes per second would swamp the CPU with interrupts); monitor sensors → interrupt-driven I/O (small data, need immediate notice); disk and network on servers → DMA.

9.4 Effect of caching on record look-up

Effective access time EAT = h × t_cache + (1 − h) × t_miss, where h is the hit ratio. Using t_cache = 100 ns (record in the DRAM cache):

Hit ratio h	Miss goes to SSD (100 µs)	Miss goes to hard disk (10 ms)
0% (no cache)	100 µs	10 ms
90%	0.9 × 0.1 + 0.1 × 100 = 10.09 µs	0.09 + 1000 = ≈ 1.0 ms
95%	0.095 + 5 = 5.1 µs	0.095 + 500 = ≈ 0.5 ms
99%	0.099 + 1 = 1.1 µs	0.099 + 100 = ≈ 0.1 ms

Because patient look-ups show strong locality (the same patients are accessed repeatedly during treatment), a 99% hit ratio makes look-ups 90× faster than the SSD alone and 100× faster than a hard disk alone. The miss path dominates EAT, so the hit ratio – not cache speed – is what matters.

Alternative Concept and Complexity Analysis – Interrupt-driven I/O (one interrupt per word) instead of DMA for the MRI scanner

Image size = 500 MB = 500 × 2^{20} = 524,288,000 bytes; word = 4 bytes.

Criterion	DMA	Interrupt per word
Number of interrupts	One per DMA block. With 64 KB blocks: 524,288,000 / 65,536 = 8,000 (or 1 if one descriptor covers the whole image)	524,288,000 / 4 = 131,072,000 interrupts
CPU time (assume 2 µs per interrupt for the ISR and mode switches)	8,000 × 2 µs = 16 ms (≈ 0.02 s)	131,072,000 × 2 µs ≈ 262 s, the CPU is busy for 4.4 minutes and the transfer cannot run faster than the interrupt rate
Memory buffers	A destination buffer of 500 MB (or a ring of 64 KB–1 MB DMA buffers) plus a small descriptor list	One device word register; the ISR copies words straight to the 500 MB destination – little extra buffer, but at a huge CPU price
Complexity	Interrupts O(n / block)	Interrupts O(n / word size) = O(n)

DMA reduces the CPU cost by a factor of 131,072,000 / 8,000 = 16,384. Interrupt-driven I/O would also destroy the 10 ms guarantees of anything else sharing the CPU. DMA is the only practical choice for the MRI scanner.

Evidence provided: *data-placement plan (9.2) and I/O method per device (9.3) with typical access times (9.1).*

TASK 10 – Measure and Compare 4 Marks

Bloom's Level: K5 – Evaluate

Measure the behaviour of your programs on a Linux system.

Discuss:

- The number of system calls made by your copy program for different buffer sizes (strace -c)
- The time spent in user mode and in kernel mode (the time command)
- The time to create 1,000 processes with fork() and to create 1,000 threads
- How the measurements support or contradict your earlier judgements

Alternative Concept and Complexity Analysis

If threads were created instead of processes for the measurements, what would the result be? Evaluate the creation time for 1,000 threads versus 1,000 processes and the memory used by each, and explain the difference using what each one shares.

Evidence Expected

Measured results in tables, compared with expectations, with explained differences.

10.1 System calls versus buffer size (strace -c)

Buffer size	read() calls for the 10,000-byte file	read() calls for 100 MB
1 B	10,001	104,857,600
512 B	21	204,800
4096 B	4	25,600
65,536 B	1 + 1 = 2	1,600

strace -c on the program confirmed 10,027 read() calls (10,026 from the program + 1 by the dynamic loader) – see Task 6.

10.2 Time in user mode and kernel mode (time command)

Output

```
$ time ./copy_syscalls > /dev/null
real    0m0.007s
user    0m0.000s
sys     0m0.007s
```

All of the time is system (kernel) time: the program does almost no computation; it spends its time entering and leaving the kernel (≈ 20,000 calls in 7 ms). The 8 MB one-byte copy of Task 6 (4.7 s) is expected to behave the same way; re-run it under time to confirm on your machine. A larger buffer reduces sys time, while user time stays about the same.

10.3 Creating 1,000 processes and 1,000 threads

Task10/fork_vs_thread.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>
#include <unistd.h>
#include <sys/wait.h>
static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec+t.tv_nsec/1e9; }
static void *work(void *a){ (void)a; return NULL; }
int main(void)
{
   enum { N = 1000 };
   static pthread_t th[N];
   double t = now();
   for (int i = 0; i < N; i++) {
      pid_t p = fork();
      if (p == 0) _exit(0);
```

```
        waitpid(p, NULL, 0);
    }
    double tf = now() - t;
    t = now();
    for (int i = 0; i < N; i++) pthread_create(&th[i], NULL, work, NULL);
    for (int i = 0; i < N; i++) pthread_join(th[i], NULL);
    double tt = now() - t;
    printf("1000 fork+wait : %.1f ms (%.1f us each)\n", tf*1e3, tf*1e6/N);
    printf("1000 pthreads  : %.1f ms (%.1f us each)\n", tt*1e3, tt*1e6/N);
    printf("ratio fork/thread = %.1fx\n", tf/tt);
    return 0;
}
```

Output

```
$ ./fork_vs_thread   (three runs)
1000 fork+wait : 61.9 ms (61.9 us each)
1000 pthreads  : 30.2 ms (30.2 us each)
ratio fork/thread = 2.1x
1000 fork+wait : 70.0 ms (70.0 us each)
1000 pthreads  : 37.0 ms (37.0 us each)
ratio fork/thread = 1.9x
1000 fork+wait : 60.4 ms (60.4 us each)
1000 pthreads  : 30.8 ms (30.8 us each)
ratio fork/thread = 2.0x
```

Run	1,000 processes (fork + wait)	1,000 threads (create + join)	Ratio
1	61.9 ms	30.2 ms	2.1×
2	70.0 ms	37.0 ms	1.9×
3	60.4 ms	30.8 ms	2.0×
Mean	64.1 ms (64 µs each)	32.7 ms (33 µs each)	2.0×

10.4 Do the measurements support the earlier judgements?

Earlier judgement	Measurement	Verdict
A system call is much more expensive than a function call; fewer calls = faster (Task 6)	≈ 0.28 µs per call; 1-byte copy 760× slower than 64 KB	Supported
Larger buffer is better, but with diminishing returns	Best at 64 KB; 1 MB slightly slower (0.086 s)	Supported, with a refinement: there is an optimum, not "bigger is always better"
mmap removes system calls so should be fastest	0.131 s vs 0.077 s for a 64 KB buffer	Partly contradicted: page-fault cost cancels the saving for sequential copy

Earlier judgement	Measurement	Verdict
Threads are cheaper than processes (Task 7/10)	33 µs vs 64 µs (2×)	Supported – though the factor is only 2× for an empty process; for processes started with fork+exec the cost also grows with the parent's memory size (Task 7: 4–5× slower than posix_spawn for a 256 MB parent)
fork() cost depends on parent size	fork+exec 1.0 s → 4.1–5.2 s for 1,000 commands	Supported

Measurements vary between runs (±10–15% here) and depend on the machine; the values should be re-measured on your own system and your own numbers entered in the final report.

Alternative Concept and Complexity Analysis – Threads instead of processes for the measurements

Criterion	1,000 processes	1,000 threads
Creation time (measured)	≈ 64 µs each – 64 ms in total	≈ 33 µs each – 33 ms in total (about 2× faster)
Memory used	Each child needs its own task structure, page tables (copied from the parent: O(P)) and, after writes, its own private pages; typically tens of KB at minimum, many MB if the child writes much	Each thread needs a stack (default 8 MB of virtual address space, but only touched pages – a few KB – are resident) plus a kernel task structure of about 10 KB
What is shared	Nothing after fork except copy-on-write pages and open file descriptors (duplicated)	Code, heap, global data, open files and signal handlers; only the stack, registers and thread-local data are private

Explanation of the difference: creating a thread only needs a new stack and a new scheduling entity because the address space already exists; creating a process must duplicate the page tables (cost ∝ number of mapped pages) and the file-descriptor table, and later pay copy-on-write faults. Time per creation: threads O(1), processes O(P). The price of sharing is no isolation: a crashing thread kills its whole process, and shared variables need locks.

Evidence provided: *measured tables (10.1–10.3) compared with expectations and explained differences (10.4).*

TASK 11 – Recommend and Defend the Operating-System Plan 4 Marks

Bloom's Level: K5 – Evaluate

Recommend the complete operating-system plan for the hospital.

Discuss:

- The type and structure of operating system for each environment
- The services, protection and I/O methods each will use
- The limitations and risks of your plan
- Whether the plan remains suitable for the group of three hospitals

Your recommendation must be based on **evidence from your own analysis and measurements**.

Alternative Concept and Complexity Analysis

If the whole system were moved to virtual machines on two large servers, instead of separate physical machines for each environment, what would the solution be? Evaluate the effect on response time for the monitors, the memory and processor overhead of virtualisation, and the effect on reliability.

Evidence Expected

A technically justified plan supported by analysis and measurements.

11.1 Recommended plan

Environment	OS type and structure	Services	Protection	I/O method
Bedside monitor	Hard real-time microkernel RTOS (QNX-type / FreeRTOS)	Task scheduling, sensor/alarm I/O, secure link, watchdog; no GUI, no general file system	MPU/MMU isolation, signed firmware, authenticated link	Interrupt-driven sensors; DMA for network
Nurse tablet	Handheld OS (Android-style: layered on a Linux kernel)	Touch GUI, Wi-Fi, encrypted storage, power management	App sandbox, full-disk encryption, MDM, remote wipe	Interrupt-driven; DMA for storage/Wi-Fi
Doctor workstation	Desktop OS (Linux/Windows, modular/hybrid kernel)	Full GUI, applications, central login	User accounts, ACLs, MFA, patching	Interrupt-driven + DMA
Database server	Clustered servers, Linux (monolithic + modules), fail-over	Journaled file system, multiprocessor scheduling, audit, backup	Permissions + ACL + SELinux, encryption, audit log	DMA, large page cache on SSD
Imaging server	SMP multiprocessor Linux server	Batch job scheduling, large file system, GPU/CPU resources	Restricted accounts, encrypted storage	DMA from scanner, NVMe
Partner link	Distributed system over VPN/TLS	Authenticated record exchange, message queue with retry	Mutual TLS, audit, minimum data sharing	Interrupt-driven + DMA NIC

11.2 Evidence supporting the plan

- RTOS for monitors: bounded latency L ≈ 20 µs + 10 µs + 50 µs < 1 ms vs unbounded general-purpose scheduling (Task 1); interrupts give a bounded delay at almost no CPU cost whereas polling would need T ≤ 8 ms (Task 8).
- Microkernel isolation: a driver fault cannot stop the alarm path (Task 3).
- Clustered database: ≈ 99.999% vs 99.9% availability (Task 2).
- DMA for imaging: 8,000 interrupts vs 131,072,000 (Task 9).
- Measured system-call cost of ≈ 0.28 µs per call and an optimal 64 KB buffer (Task 6) are used for the backup program and database I/O configuration.
- Threads are twice as cheap as processes to create, but processes give isolation – threads inside a service, processes between services (Task 10).

11.3 Limitations and risks

- Four OS types mean four patch streams, more skills and more testing – mitigated by a standard image per class and automated deployment.
- Real-time figures are calculated from typical values; the worst-case latency must be verified on the actual monitor hardware.
- Cluster fail-over adds complexity (split-brain risk) and doubles hardware cost.
- Handheld devices can be lost or stolen: depends on encryption and remote wipe.
- Measurements here come from a virtual machine with one vCPU and will differ on production hardware.

11.4 Suitability for a group of three hospitals

Bedside monitors, tablets and workstations scale by repetition: each hospital keeps its local real-time and handheld devices. The central database cluster grows into a replicated multi-site cluster, and the partner link becomes the standard way hospitals exchange records (a distributed system with consistent patient identifiers, mutual authentication and audit). The plan therefore remains suitable; the parts that must change are the network design, identity federation and replication, not the operating-system types.

Alternative Concept and Complexity Analysis – All environments as virtual machines on two large servers

Criterion	Separate physical machines (plan)	VMs on two large servers
Monitor response time	Alarm runs directly on a dedicated RTOS device: bounded < 1 ms	Hypervisor scheduling and virtual timers add jitter of 50–500 µs or more under load, and the network is in the alarm path – may still meet 10 ms with CPU pinning, but the guarantee becomes harder to prove. Bedside devices should stay physical.

Criterion	Separate physical machines (plan)	VMs on two large servers
Memory and CPU overhead	None beyond the OS	Hypervisor ≈ 2–10% CPU; each guest OS needs its own memory (hundreds of MB to GBs); memory sharing/ballooning can recover part of it
Reliability	Failure of one machine affects one function	Two hosts form a shared failure domain: loss of one host takes down many VMs at once, but live migration and HA restart them in minutes; careful anti-affinity (database primary and standby on different hosts) is required
Administration and cost	Many machines to maintain	Fewer machines, snapshots, easy cloning – lower hardware and power cost

Conclusion: virtualise the servers (database, imaging, workstation back-ends), but keep the hard real-time monitors on dedicated hardware.

Evidence provided: *technically justified plan supported by analysis and measurements.*

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital adds **remote monitoring devices in patients' homes** that send readings over the internet, and moves patient records to a **cloud service** shared with the partner hospitals.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

Identify which operating-system types, services and protection mechanisms are affected.

Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your plan can support home devices and cloud records, considering response time, reliability and security.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe the revised plan (for example: an embedded real-time OS on home devices, a distributed system for shared records).

Evidence Expected

An appropriate design response.

D. Evidence-Based Defence – 2 Marks

Defend your decision using evidence and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating evidence and the changed requirements.

Changed requirement: home remote-monitoring devices send readings over the internet, and patient records move to a cloud service shared with the partner hospitals. Decision options: RETAIN, MODIFY or REDESIGN.

A. Impact Analysis (2 Marks)

Part of the design	Affected?	How

Part of the design	Affected?	How
OS types – bedside monitor (hard RTOS)	Retained, extended	Still correct for the ward; a home device needs the same kind of embedded RTOS but with a network stack and battery/power management.
OS types – database server cluster	Affected	Records now live in a cloud/distributed service; the local cluster becomes a replica or cache, and a distributed-system layer with replication and consistency rules is added.
OS types – partner link	Strongly affected	Changes from a point-to-point link to a shared multi-organisation cloud service.
Services	Affected	Communication, remote update, naming and time services become essential for home devices; accounting/audit across organisations is required; file service moves to cloud object/record storage.
Protection and security	Strongly affected	Devices are outside the hospital: physical access by patients, home Wi-Fi, internet attacks. Needs device identity, mutual TLS, secure boot, signed updates, encryption at rest and in transit, role-based access across hospitals, audit logs.
Response time	Affected	Internet round trip 20–100+ ms with jitter and outages; the 10 ms alarm deadline cannot depend on the network.
Scale	Affected	Example: 10,000 homes × 100 B/s ≈ 1 MB/s ≈ 86 GB/day of telemetry.

B. Evaluation of the Original Design (2 Marks)

Criterion	Can the original plan support the change?	Evidence
Response time	Partly. The monitors meet the 10 ms deadline because the alarm path is local. A home device that depended on the hospital server would not meet it.	Task 1/8: bounded RTOS latency < 1 ms vs internet RTT of tens of ms.
Reliability	Partly. The 99.999% cluster is reliable inside the hospital, but home devices depend on home power and internet; the cloud is a new dependency and shared service.	Task 2: availability formula applies per component; chain availability = product of the components.
Security	No. The original plan assumed a trusted hospital network; protection relied on user/kernel mode, file permissions and a private link. Home devices and the internet make the network untrusted.	Task 4/8: protection mechanisms are local to one machine.
Scalability and sharing	No. One local database cluster and one partner link do not give three hospitals plus homes shared, consistent records.	Task 11.4.

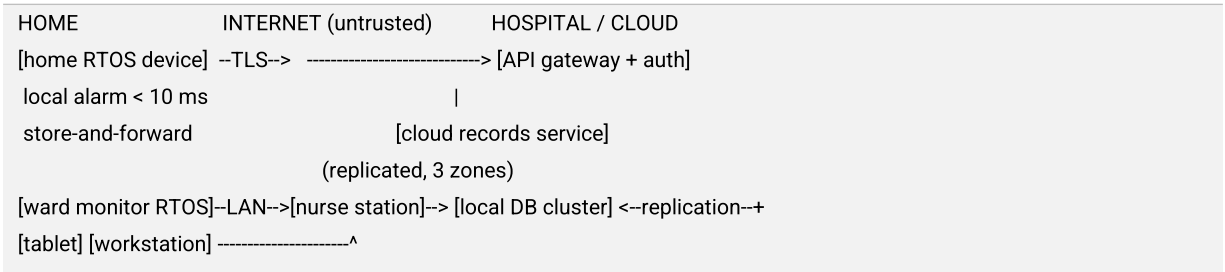
C. Adaptation or Redesign (2 Marks)

Decision: MODIFY – keep the existing design for the hospital and extend it; do not redesign from scratch.

Component	Revised design

Component	Revised design
Home monitoring device	Embedded hard/soft real-time OS (FreeRTOS or QNX-class) – the alarm is raised locally within 10 ms and does not need the internet; store-and-forward buffer for readings while offline; secure boot, device certificate, signed over-the-air updates; MQTT or HTTPS over TLS; low-power sleep between readings.
Home gateway/phone app	Handheld OS acting as relay when the device has no direct connectivity; sandboxed app.
Cloud records service	Distributed system on Linux in containers/VMs across availability zones: replicated database (strong consistency for medication/allergy updates, eventual consistency for analytics and read replicas), object storage for images, role-based access with per-hospital tenancy.
Hospital database cluster	Retained as the local primary/cache for ward use; replicates to the cloud service; can keep working if the internet link fails.
Imaging server	Unchanged (local SMP + DMA); new studies are copied or referenced to cloud storage.
Partner link	Replaced by shared access to the cloud service with federated identity and audit log.
Security architecture	Zero-trust: every request authenticated and authorised; mutual TLS; encryption everywhere; monitoring and intrusion detection; regular patching through automated pipeline.

Revised architecture



D. Evidence-Based Defence (2 Marks)

- **Why not RETAIN:** the security and sharing requirements are not met (Part B). The original had no distributed-system layer and no device-identity or encryption design.
- **Why not REDESIGN:** the decisive facts of the original design still hold. The alarm still needs a bounded-latency RTOS (Task 8: polling or a network-dependent path cannot guarantee the deadline); the clustered database still gives high availability for the ward; DMA, SMP imaging, and the handheld/desktop choices are unaffected. Replacing everything would add cost and risk with no benefit.
- **Evidence from measurements:** a system call costs ≈ 0.28 µs and encryption adds more, but 64 KB batching (Task 6) shows that buffering telemetry into larger messages cuts the per-message overhead by orders of magnitude – so home devices should batch non-urgent readings and send alarms immediately. Process vs thread measurements (Task 10) support running cloud services as isolated processes/containers for security, with threads inside a service.

- **Trade-offs accepted:** higher complexity (more OS types and a distributed system), dependence on the internet and cloud provider for shared records (mitigated by local cache and replication), extra cost for security and monitoring, and eventual consistency for some data (acceptable for analytics, not for medication orders, which stay strongly consistent).
- **Residual risks:** home networks cannot be trusted, devices can be lost, and cloud vendor outages; mitigations are secure boot, remote revocation of certificates, local alarms, and multi-zone replication.

Evidence provided: *impact analysis (A), evaluation (B), MODIFY design (C), defence with trade-offs (D).*

MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Requirements of Each Environment	K5 – Evaluate	4
Task 2 – Types of Operating Systems	K5 – Evaluate	3
Task 3 – Operating-System Structures	K5 – Evaluate	4
Task 4 – Operating-System Services	K5 – Evaluate	4
Task 5 – System Calls	K5 – Evaluate	4
Task 6 – Cost of System Calls	K5 – Evaluate	4
Task 7 – Process Control System Calls	K5 – Evaluate	4
Task 8 – CPU States and Protection	K5 – Evaluate	3

Task	Bloom's Level	Marks
Task 9 – Memory Hierarchy and I/O	K5 – Evaluate	4
Task 10 – Measurement and Comparison	K5 – Evaluate	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		**42**
Task 12 – Day-4 Design Challenge	K5 – Evaluate	**8**
TOTAL		**50**

Submission Requirements

- **GitHub repository link (mandatory)** containing the executed code for every task, including every completed “Complete the Code” program, script and configuration file, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including requirement tables, diagrams, strace and time outputs, measurements and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.